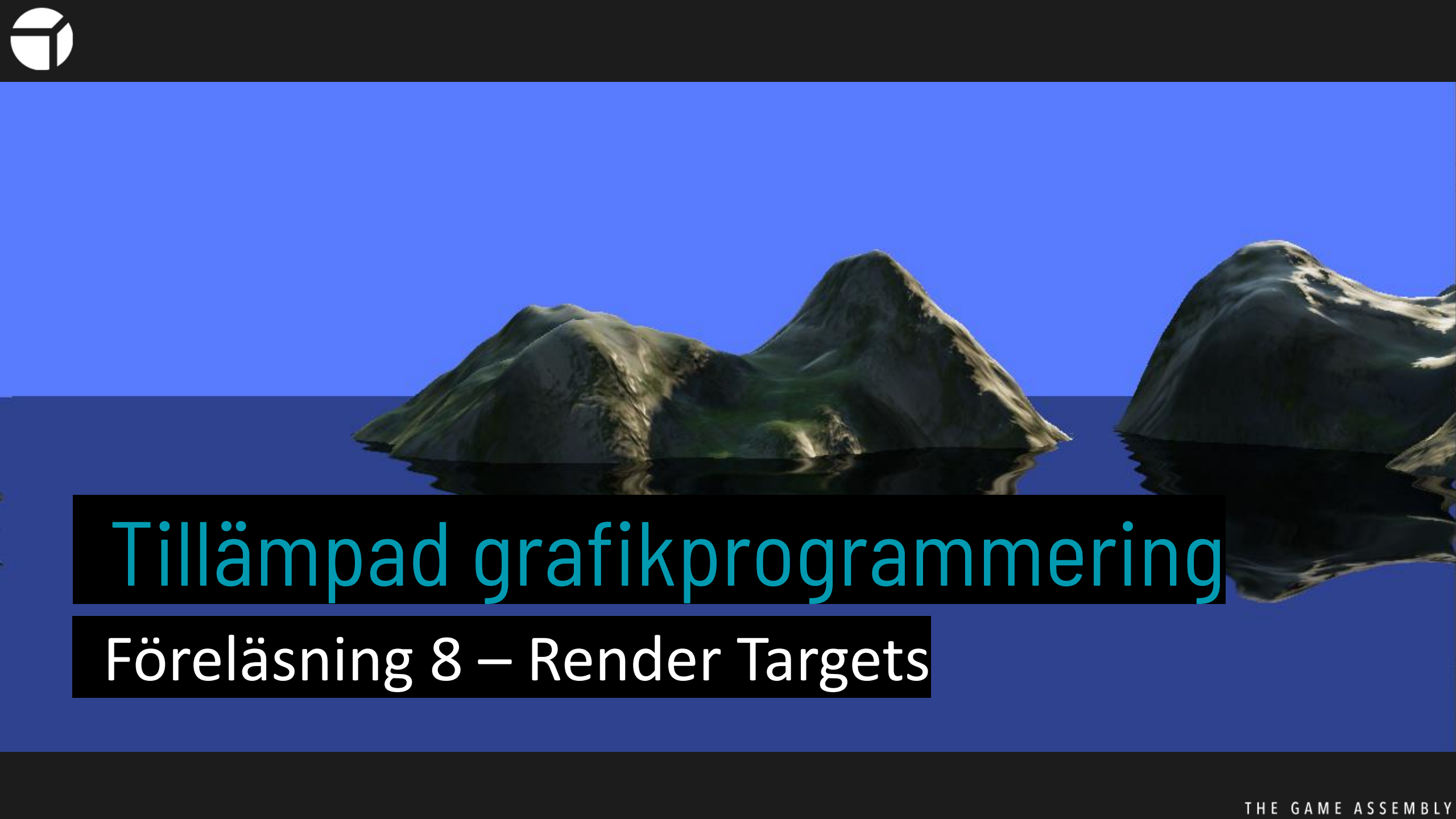




Tillämpad grafikprogrammering

Föreläsning 8 – Render Targets



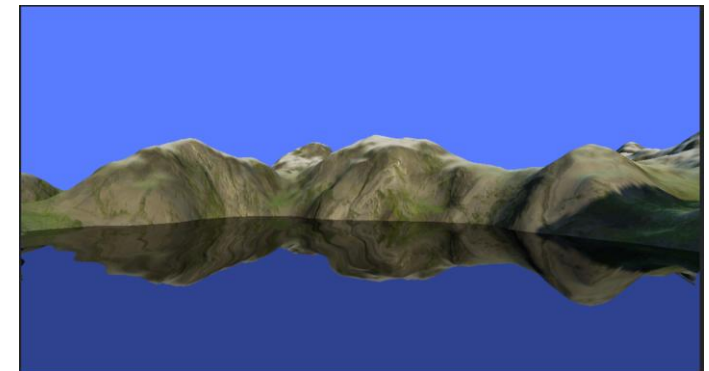
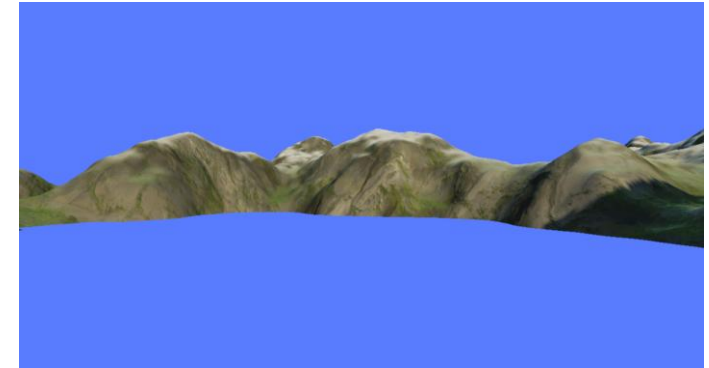
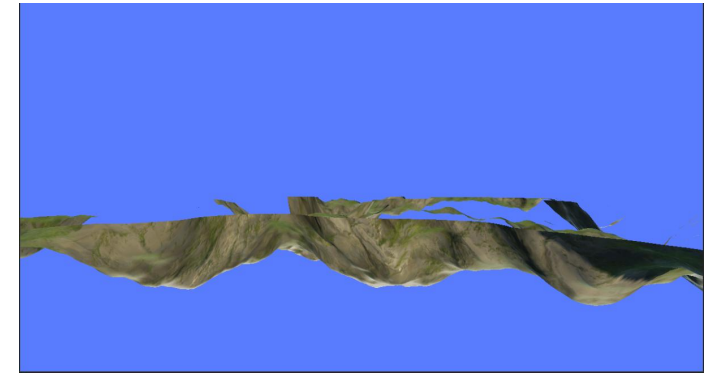
- Vi har hittills bara ritat till backbuffern
 - Men man kan rita till texturer också!
 - Dessa kan sen används som texturer i en pixel shader.
- Sista riktigt viktiga byggstenen
- Grunden i många avancerade tekniker
 - Vi ska prata om två!



Planar reflections



- Rita världen upp och ner till en textur
 - Undvik att rita allt “ovanför” vattenytan
- Rita världen som vanligt
 - Till backbuffern
- Rita ett plan
 - Läs från reflektionstexturen
 - Offsetta lite för att få vågor





- Hur ritar vi världen upp och ner?

- Skala en axel med -1

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Vattenhöjd h

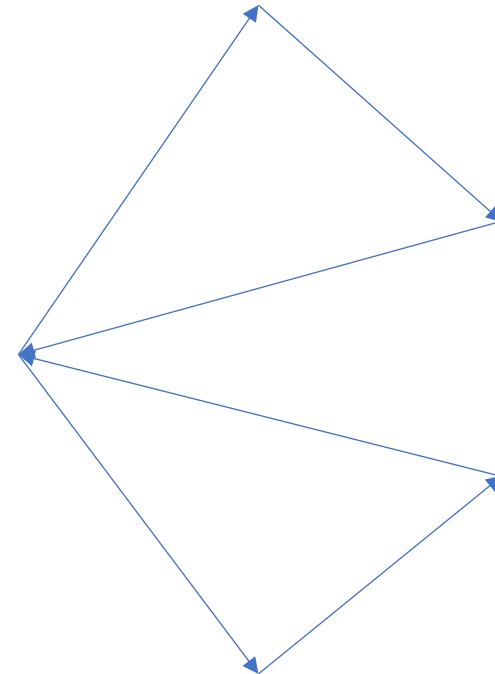
- Flytta koordinatsystemet
- Spegla
- Flytta tillbaka

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & h & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -h & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -2h & 0 & 1 \end{bmatrix}$$



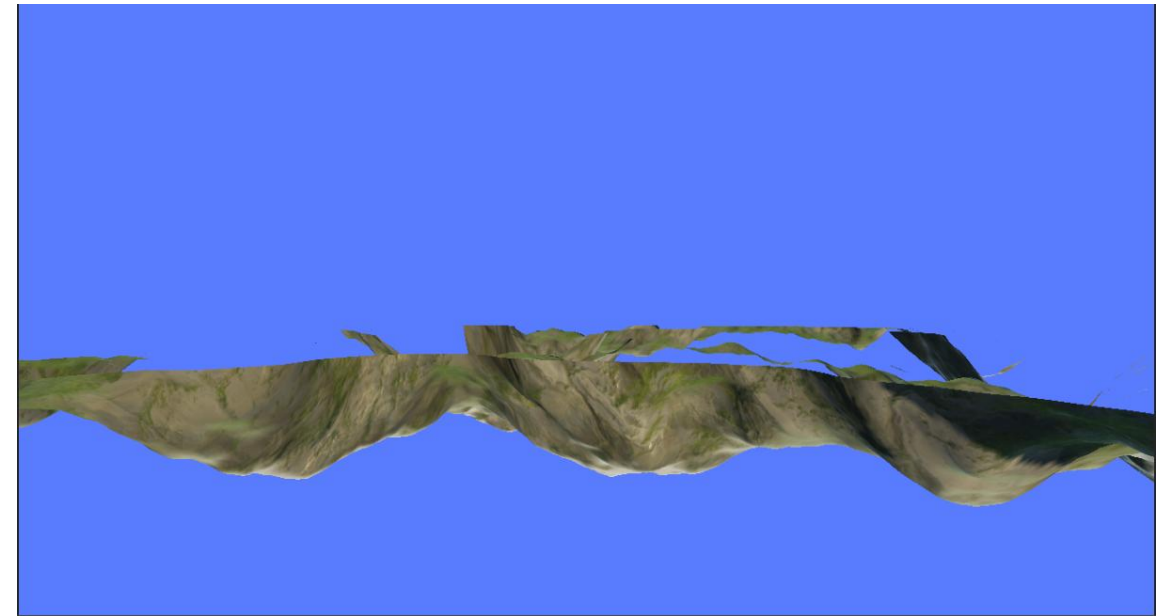
- Vi ritar med backface culling
 - Kollar om triangeln är med eller moturs
- Vad händer när vi speglar?
- Måste byta till frontface culling!





- Kan lägga till en *SV_ClipDistance* och sätta i vertex shadern
 - Ritar bara delar av trianglar när *SV_ClipDistance* > 0
- Sätt *SV_ClipDistance* till *position.y - h*
- Då kapas allt under höjd *h*!

```
struct TerrainPixelInputType
{
    float4 position : SV_POSITION;
    float4 worldPosition : POSITION;
    float4 worldNormal : NORMAL;
    float3 worldTangent : TANGENT;
    float3 worldBinormal : BINORMAL;
    float2 uv : TEXCOORD0;
    float clip : SV_ClipDistance0;
};
```





- Hur använder vi normalerna för få lite distortion?
 - Går inte att göra riktigt rätt
 - Alla spel med planar reflections fuskar!
- Ett enkelt sätt
 - Räkna ut normalen på vattenytan, relativt kameran
 - Offsetta uv-koordinaterna med normal.xz
 - skalat med avståndet till kameran och en valfri konstant



Nu mer praktiskt!



```
struct RenderTarget
{
    ComPtr<ID3D11RenderTargetView> renderTargetView;
    ComPtr<ID3D11ShaderResourceView> shaderResourceView;
};
```



```
HRESULT result;
D3D11_TEXTURE2D_DESC desc = { 0 };
desc.Width = textureDesc.Width;
desc.Height = textureDesc.Height;
desc.MipLevels = 1;
desc.ArraySize = 1;
desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
desc.SampleDesc.Count = 1;
desc.SampleDesc.Quality = 0;
desc.Usage = D3D11_USAGE_DEFAULT;
desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
desc.CPUAccessFlags = 0;
desc.MiscFlags = 0;

ID3D11Texture2D* texture;
result = myDevice->CreateTexture2D(&desc, nullptr, &texture);
assert(SUCCEEDED(result));

result = myDevice->CreateShaderResourceView(texture, nullptr, &myWaterReflectionRenderTarget.shaderResourceView);
assert(SUCCEEDED(result));

result = myDevice->CreateRenderTargetView(texture, nullptr, &myWaterReflectionRenderTarget.renderTargetView);

texture->Release();
```



```
D3D11_RASTERIZER_DESC rasterizerDesc = {};  
rasterizerDesc.CullMode = D3D11_CULL_FRONT;  
rasterizerDesc.FillMode = D3D11_FILL_SOLID;  
  
result = myDevice->CreateRasterizerState(&rasterizerDesc, &myFrontFaceCullingRasterizerState);  
if (FAILED(result))  
    return false;
```



- Byta till och tömma vår nya render target:

```
myContext->OMSetRenderTargets(1, myWaterReflectionRenderTarget.renderTargetView.GetAddressOf(), myDepthBuffer.Get());  
myContext->ClearRenderTargetView(myWaterReflectionRenderTarget.renderTargetView.Get(), color);  
myContext->ClearDepthStencilView(myDepthBuffer.Get(), D3D11_CLEAR_DEPTH | D3D11_CLEAR_STENCIL, 1.0f, 0);  
myContext->RSSetState(myFrontFaceCullingRasterizerState.Get());
```

- Byta tillbaka och tömma:

```
myContext->OMSetRenderTargets(1, myBackBuffer.GetAddressOf(), myDepthBuffer.Get());  
myContext->ClearRenderTargetView(myBackBuffer.Get(), color);  
myContext->ClearDepthStencilView(myDepthBuffer.Get(), D3D11_CLEAR_DEPTH | D3D11_CLEAR_STENCIL, 1.0f, 0);  
myContext->RSSetState(nullptr);
```

- Obs! Viktigt att vi tömmer depthbuffern, eftersom den återanvänds



- Utan normal-offset för reflektionerna:

```
float fresnel = Fresnel_Schlick(  
    float3(0.25f, 0.25f, 0.25f),  
    float3(0.0f, 1.0f, 0.0f),  
    toEye)  
  
float3 reflection = aTexture.Sample(aSampler, input.position.xy/resolution).rgb;  
  
result.color.rgb = fresnel * reflection;  
result.color.a = 1.f;  
  
return result;
```



- Valfritt: lägg till en offset till vågorna så de rör sig
 - Ganska pilligt med koordinatsystemen, därför valfritt
 - Mina uträkningar är inte helt korrekt, men ger OK resultat

```
float4x4 cameraTranspose = transpose(cameraMatrix);  
float3 toEye = cameraTranspose[3].xyz - input.worldPosition.xyz;  
float dist = abs(dot(toEye, cameraTranspose[2].xyz));
```

```
float2 p = input.worldPosition.xz;  
float2 k0 = float2(6.f, 16.f);  
float2 k1 = float2(10.f, -14.f);  
float A = 0.0005f;
```

```
// derivative with respect to x and z of height sin(dot(p, k0) + time) + sin(dot(p, k1) + time)  
float2 heightDerivative = k0 * sin(dot(p, k0) + time) + k1 * sin(dot(p, k1) + time);  
float2 maxValue = float2(0.f, length(k0) + length(k1));
```

```
// adding max values of heightDerivative to not sample to high  
// dividing by dist to make waves smaller at distance  
// scaling by Amplitude parameter  
float2 offset = A * (maxValue + heightDerivative) / dist;
```

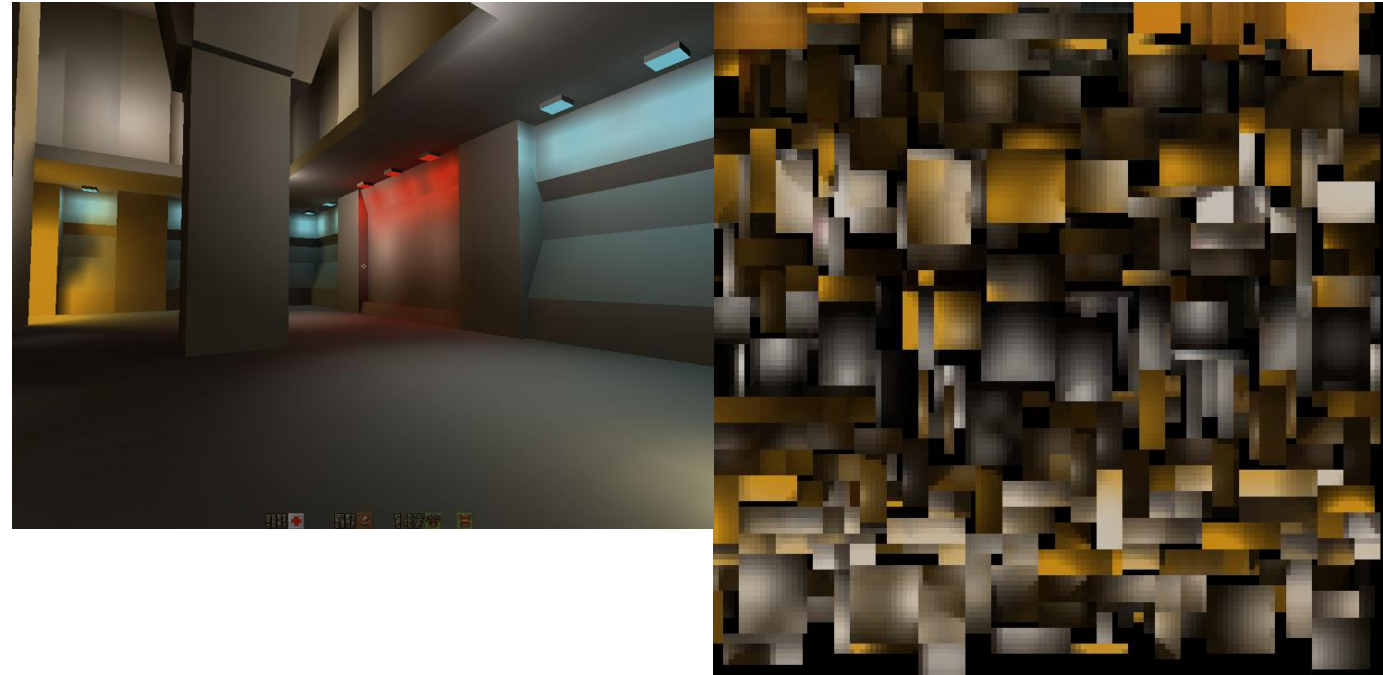
```
float3 reflection = aTexture.Sample(aSampler, input.position.xy/resolution + offset).rgb;
```



Lightmaps



- Recap från sist
- Light maps
 - Sparar ljus i en textur
 - Alla trianglar måste ha unika UVs!
 - Använts sen 90-talet
- Kan spara olika saker:
 - Komplette ljussättning
 - Bara studsat ljus
 - Ambient Occlusion
 - Bara skuggor
- Avvägning
 - Hur hög upplösning behövs?
 - Är ljussättningen dynamisk?
 - Är normal maps viktigt?

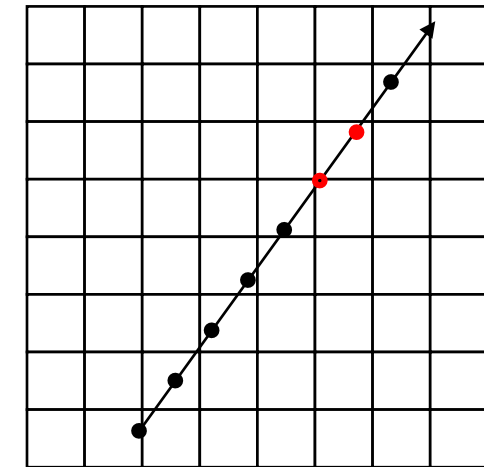
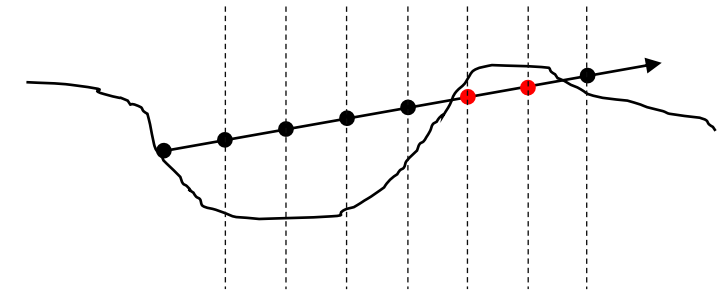




- Detta kommer vara en VG-uppgift, så går igenom i mindre detalj
- Kommer vara del av uppgiften att hitta hur man gör exakt
- Vill göra följande:
 - Räkna ut en lightmap för vår terräng
 - Uträkningen ska ske när programmet startar
 - Uträkningen ska göras i en pixel shader
 - Lightmappen ska
 - Ha formatet DXGI_FORMAT_R32G32_FLOAT
 - R ska innehålla Ambient Occlusion
 - G ska innehålla om en pixel är i skugga eller ej
 - När vi ritar terrängen:
 - Skala directional light med skugg-informationen
 - Skala cubemappen med ambient occlusion-information



- Hur gör vi det?
- Vi skriver en lite raytracer i vår shader!
 - Den behöver kunna läsa heightmap och normal
 - För Ambient Occlusion
 - vi strålar åt alla håll, och räknar hur många som inte träffar mark
 - Exempelkod finns i valfria raytracer-delen av linalg
 - Directional Light
 - som i Raytracern från Linjär Algebran
- Enda vi kan kollidera mot är terrängen
 - Ta små steg, läs terränghöjden
 - Är vi under... kollision!





- Loopar i HLSL

- Ser ut som C++
 - for, while, do while
- Men!
 - Man kan styra hur loopar kompileras
 - Tex kan man tvinga kompilatorn att unrolla med [unroll(8)]

- Vi kommer använda

- [fastopt]
- Kan minska kompileringstiderna drastiskt!

```
[fastopt]    |  
for (int i = 0; i < 100; i++)
```

- <https://docs.microsoft.com/en-us/windows/win32/direct3dhls/dx-graphics-hlsl-for>



- Texturer och mipmappar
 - När vi tidigare läst texturer later vi shadern automatiskt välja mipmap
 - `aTexture.Sample(sampler, uv)`
- Hur väljer den mipmap?
 - Den kollar på pixlarna brevid, och ser var de läser texturen
 - Alla 2x2 pixlar behöver därför köra samma kod!
- Nu vill vi ha dynamiska loopar
 - Kan då köra
 - `aTexture.SampleLevel(sampler, uv, mipMapLevel)`
 - `mipMapLevel = 0` för högsta
- Gör ni inte detta få ni en varning och långa kompileringstider

